Emirates ERP — Production-Ready Multi-Tenant SaaS Architecture

> **Document Version:** 1.0
> **Classification:** Internal Technical Reference
> **Audience:** Software Engineers, Architects, DevOps Teams
> **Stack:** Node.js · PostgreSQL · Redis · JWT · Docker · Kubernetes

---

Executive Summary

Emirates ERP is a multi-tenant SaaS platform that serves thousands of organizations across HR, Finance, Supply Chain, Fleet, and CRM modules. This document defines a production-grade, horizontally scalable, secure, and maintainable architecture capable of serving millions of users with strong tenant isolation, sub-100ms response times under typical load, and 99.99% availability.

The architecture follows a **shared-database, schema-per-tenant** model, combined with a stateless API layer, Redis-backed caching, JWT authentication with RBAC, and asynchronous background workers via BullMQ.

Part 1: Analysis of Existing Architecture

1.1 What the Current Architecture Is Doing

Based on the observed frontend codebase (`Login.jsx` , `TenantContext` , `tenantUtils.js`), the current Emirates ERP system:

- Identifies tenants via **subdomain** (`getTenantFromSubdomain()`)
- Authenticates using **JWT tokens** stored in `localStorage`
- Implements **role-based routing** (SUPER_ADMIN, MASTER_ADMIN, HRMS_ADMIN, POS_ADMIN, etc.)
- Stores modules and themes per-tenant in `localStorage`
- Communicates with a REST API via a single `VITE_API_BASE_URL` environment variable
- Uses **axios** for HTTP requests with Bearer token headers

1.2 Strengths

Strength	Detail
Subdomain-based tenancy	Clean, URL-native isolation
Role-based navigation	Separation of concerns in routing
Theme-per-tenant	Good UX customization support
JWT authentication	Stateless, scalable
Module access control	Tenant-specific feature gating

1.3 Weaknesses

Weakness	Risk
JWT in `localStorage`	XSS vulnerability — tokens exposed to scripts
Single `API_BASE_URL`	No gateway layer, no failover
No token refresh observed	Token expiry causes hard logouts
Roles stored as raw strings	Brittle, not normalized
No rate limiting visible	Susceptible to brute force
Modules stored as raw string in localStorage	Not validated client-side

1.4 Scalability Issues

- Single API endpoint (no load balancing at config level)
- No caching layer evident (every request hits the backend)
- No mention of connection pooling or read replicas
- Role checks likely occur in every request without caching

1.5 Security Issues

- `localStorage` JWT storage is vulnerable to XSS
- No CSRF protection observable
- Module permissions not re-validated server-side per request
- Missing JWT rotation/blacklist mechanism

1.6 Multi-Tenant Limitations

- Tenant identity tied only to subdomain; no fallback for header-based or custom domain tenancy

- No evident schema-level isolation in the database
- No tenant-level rate limiting

1.7 Performance Bottlenecks

- Permission checks on every request without Redis caching
- No CDN or static asset optimization visible
- No observable query optimization or pagination strategy

1.8 Database Concerns

- Unknown indexing strategy
- No observable multi-tenant partitioning
- Risk of cross-tenant data leakage without Row-Level Security (RLS)

1.9 Cache Strategy Concerns

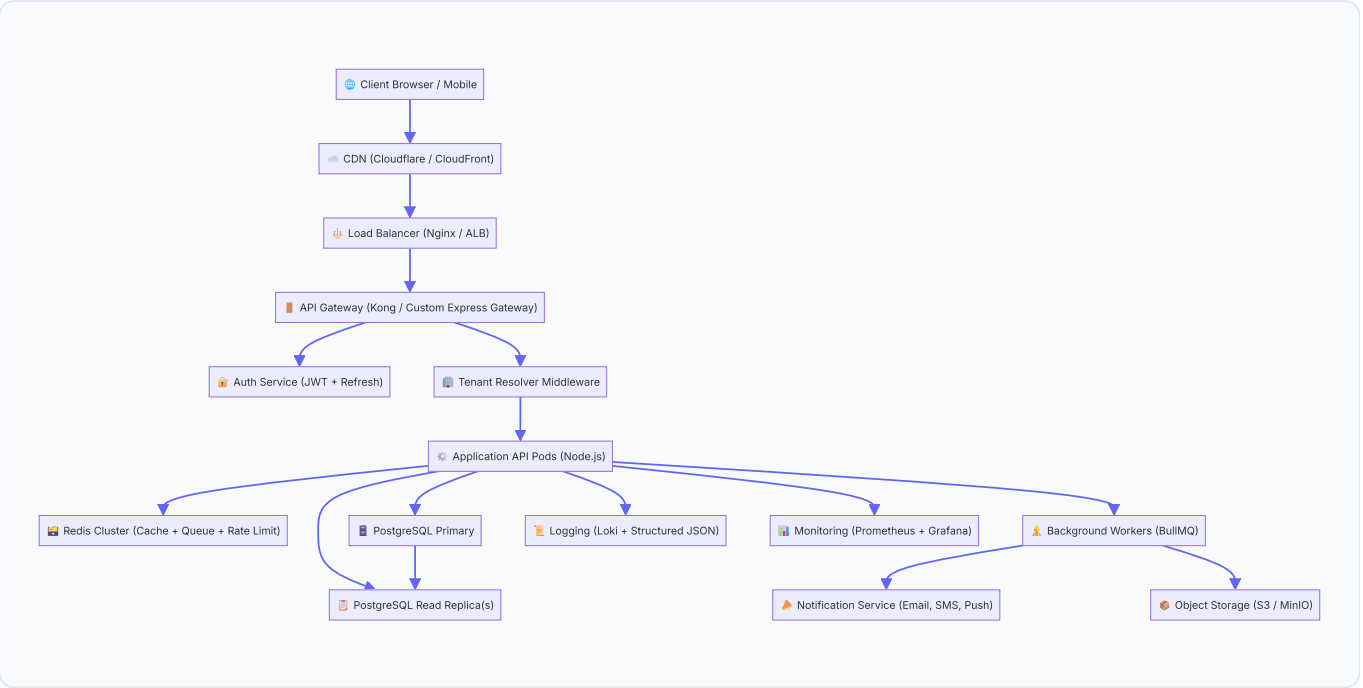
- No Redis caching observed in frontend data flows
- JWT blacklisting mechanism not apparent
- Tenant metadata fetched fresh on every session

1.10 Suggested Improvements Summary

1. Move JWT to `HttpOnly` cookies
2. Implement Redis-backed permission and tenant caching
3. Add an API Gateway (Kong or custom Express gateway)
4. Implement schema-per-tenant isolation in PostgreSQL
5. Add BullMQ for async jobs (email, audit, reports)
6. Enable Row-Level Security (RLS) in PostgreSQL
7. Add rate limiting middleware with Redis
8. Implement refresh token rotation with Redis blacklist

Part 2: Production Multi-Tenant Architecture

2.1 High-Level Architecture



Component Communication:

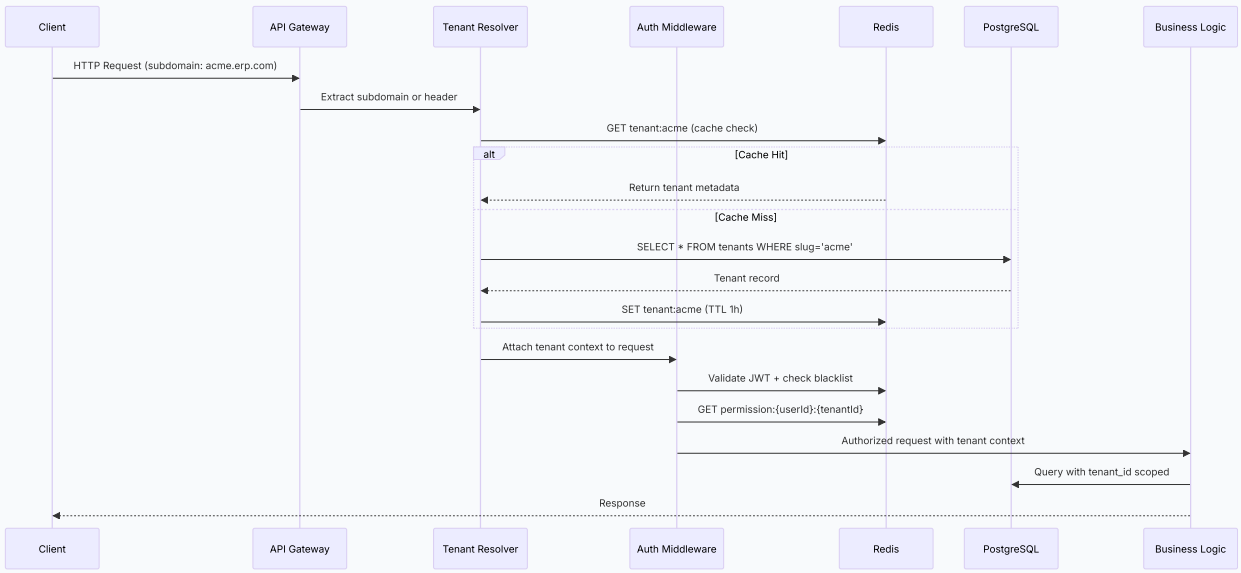
From	To	Protocol
API Gateway	Auth Service	Internal HTTP / gRPC
API Gateway	App Pods	HTTP/2
App Pods	Redis	Redis Protocol (TCP)
App Pods	PostgreSQL	TCP + Connection Pool (PgBouncer)
App Pods	BullMQ Workers	Redis Queue
Workers	Notification Service	Internal HTTP
Workers	Object Storage	S3 API

2.2 Tenant Resolution Flow

Identification Strategies (in priority order):

1. **Subdomain** → `company.app.emirateserp.com` → `tenantId = "company"`
2. **Custom Domain** → Reverse-lookup via Redis cache or DB
3. **Request Header** → `X-Tenant-ID: company`
4. **JWT Claim** → `tenant_id` embedded in JWT payload

Request Lifecycle:



2.3 Database Design Strategy

Recommendation: Shared Database — Schema Per Tenant

Criterion	Shared Schema	Schema Per Tenant ✅	DB Per Tenant
Isolation	Low	Medium-High	Highest
Cost	Lowest	Low	Highest
Scaling	Hard	Medium	Easy
Migration	Risky	Manageable	Complex
Backup	Shared	Per-schema	Per-DB
Connection Pool	Shared	Shared	Separate per tenant
Query Complexity	Row filter needed	Schema isolation	Simple

Why Schema Per Tenant for Emirates ERP:

- Thousands of tenants → DB-per-tenant is too expensive
- Compliance/isolation needs → shared schema is risky
- Schema-per-tenant gives logical isolation with manageable operational overhead
- PostgreSQL natively supports multiple schemas within one database

- Row-Level Security (RLS) adds an extra safety net

2.4 PostgreSQL Schema Design

Master Schema (public)

```
-- Tenants Registry
CREATE TABLE public.tenants (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    slug        VARCHAR(100) UNIQUE NOT NULL,   -- subdomain key
    name        VARCHAR(255) NOT NULL,
    plan        VARCHAR(50) DEFAULT 'standard', -- free|standard|enterprise
    status      VARCHAR(20) DEFAULT 'active',   -- active|suspended|deleted
    schema_name VARCHAR(100) UNIQUE NOT NULL,   -- e.g., tenant_acme
    settings    JSONB DEFAULT '{}',
    created_at  TIMESTAMPTZ DEFAULT NOW(),
    updated_at  TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_tenants_slug ON public.tenants(slug);
CREATE INDEX idx_tenants_status ON public.tenants(status);

-- Custom Domains
CREATE TABLE public.tenant_domains (
    id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON DELETE CASCADE,
    domain    VARCHAR(255) UNIQUE NOT NULL,
    verified  BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_tenant_domains_domain ON public.tenant_domains(domain);

-- Subscriptions
CREATE TABLE public.subscriptions (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id   UUID NOT NULL REFERENCES public.tenants(id),
    plan        VARCHAR(50) NOT NULL,
    status      VARCHAR(20) DEFAULT 'active',
    starts_at   TIMESTAMPTZ NOT NULL,
    expires_at  TIMESTAMPTZ,
    meta        JSONB DEFAULT '{}'
);
```

Per-Tenant Schema (tenant_{slug})

```
-- Template schema created for each tenant on provisioning

-- Users
CREATE TABLE users (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id    UUID NOT NULL,  -- redundant but useful for RLS
    email        VARCHAR(255) UNIQUE NOT NULL,
    username     VARCHAR(100),
    password_hash TEXT NOT NULL,
    status       VARCHAR(20) DEFAULT 'active',
    last_login   TIMESTAMPTZ,
    created_at   TIMESTAMPTZ DEFAULT NOW(),
    updated_at   TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_tenant ON users(tenant_id);

-- Roles
CREATE TABLE roles (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name        VARCHAR(100) NOT NULL,
    description TEXT,
    is_system   BOOLEAN DEFAULT FALSE,  -- built-in vs custom roles
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Permissions
CREATE TABLE permissions (
    id       UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    module   VARCHAR(100) NOT NULL,  -- hr | finance | pos | crm | fleet
    action   VARCHAR(100) NOT NULL,  -- read | write | delete | admin
    resource VARCHAR(100),           -- optional fine-grained resource
    UNIQUE(module, action, resource)
);

-- Role-Permission Mapping
CREATE TABLE role_permissions (
    role_id       UUID NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
    permission_id UUID NOT NULL REFERENCES permissions(id) ON DELETE CASCADE,
    PRIMARY KEY (role_id, permission_id)
);

-- User-Role Membership
CREATE TABLE user_roles (
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    role_id UUID NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
    granted_by UUID REFERENCES users(id),
    granted_at TIMESTAMPTZ DEFAULT NOW(),
    PRIMARY KEY (user_id, role_id)
);

-- Audit Logs (partitioned by month)
CREATE TABLE audit_logs (
    id          UUID DEFAULT gen_random_uuid(),
    user_id     UUID,
    action      VARCHAR(200) NOT NULL,
```

```
    entity_type VARCHAR(100),
    entity_id   UUID,
    ip_address  INET,
    user_agent  TEXT,
    meta        JSONB DEFAULT '{}',
    created_at  TIMESTAMPTZ DEFAULT NOW()
) PARTITION BY RANGE (created_at);

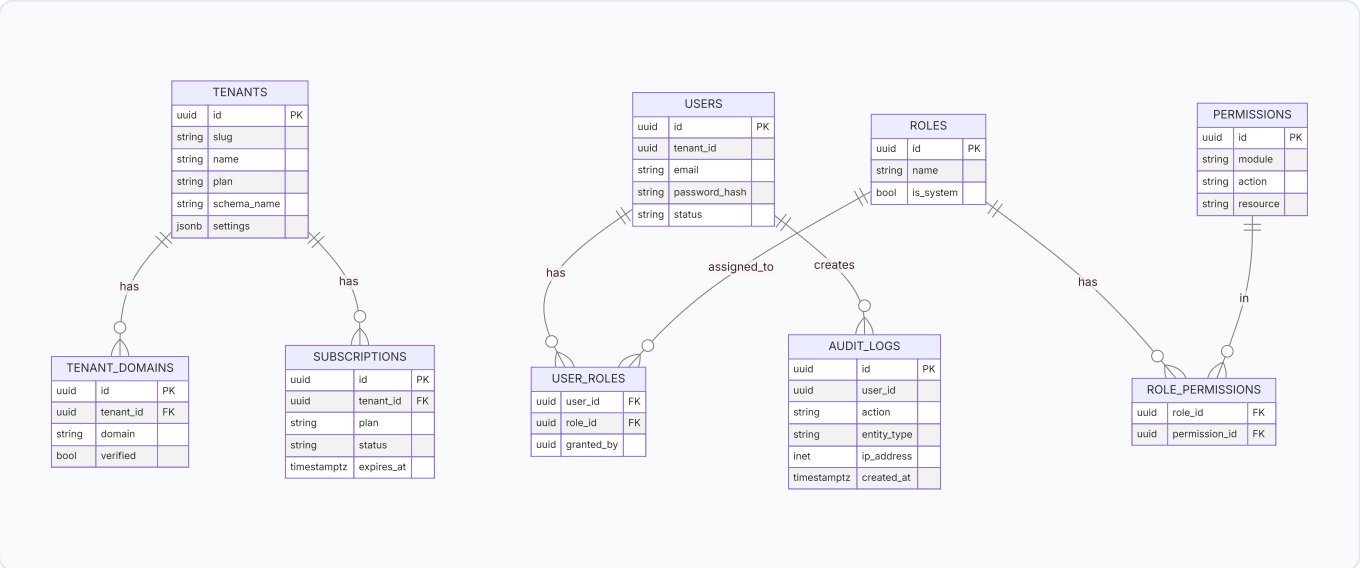
-- Create monthly partitions (automate with pg_partman)
CREATE TABLE audit_logs_2026_01 PARTITION OF audit_logs
    FOR VALUES FROM ('2026-01-01') TO ('2026-02-01');

-- Settings
CREATE TABLE settings (
    key        VARCHAR(200) PRIMARY KEY,
    value      JSONB NOT NULL,
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Modules / Feature Flags
CREATE TABLE module_access (
    module     VARCHAR(100) NOT NULL,
    enabled    BOOLEAN DEFAULT TRUE,
    config     JSONB DEFAULT '{}',
    PRIMARY KEY (module)
);

-- Row-Level Security
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON users
    USING (tenant_id = current_setting('app.current_tenant_id')::UUID);
```

Entity Relationship Diagram:



2.5 Redis Design

Redis Usage Map:

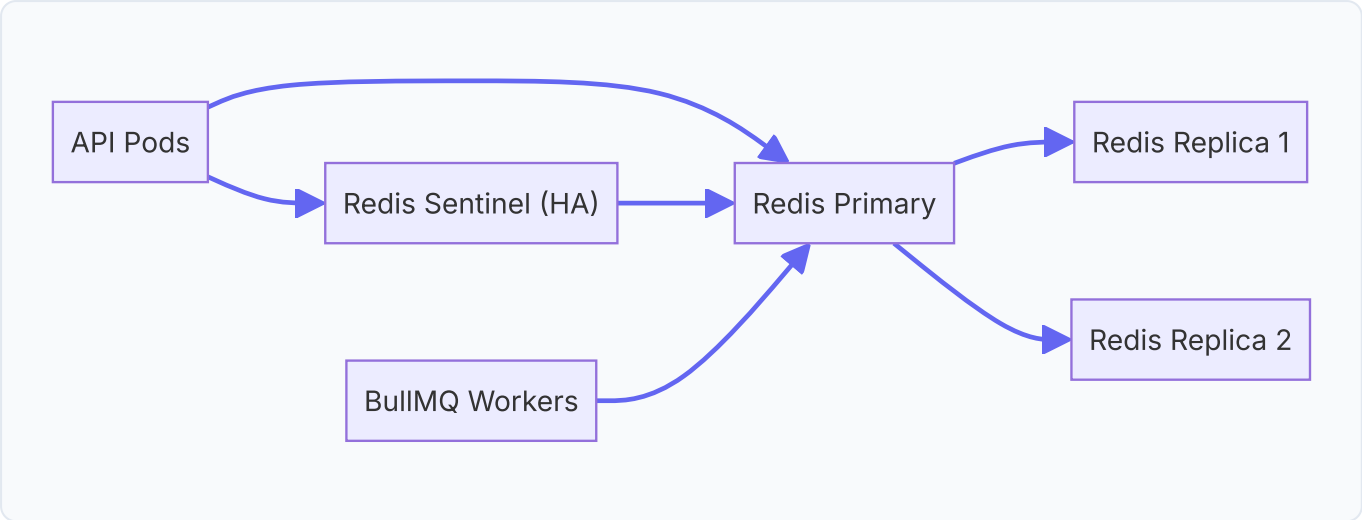
Use Case	Key Pattern	TTL	Type
Tenant metadata	`tenant:{slug}`	1 hour	Hash
User session	`session:{sessionId}`	24 hours	Hash
JWT blacklist	`jwt:blacklist:{jti}`	Token TTL	String
Refresh token	`refresh:{userId}:{tokenId}`	7 days	String
User permissions	`perm:{userId}:{tenantId}`	30 min	JSON String
Rate limit	`ratelimit:{ip}:{route}`	1 min	Counter
Tenant rate limit	`ratelimit:tenant:{tenantId}`	1 min	Counter
Feature flags	`flags:{tenantId}`	5 min	Hash
API response cache	`cache:{tenantId}:{route}:{hash}`	5 min	String
Distributed lock	`lock:{resource}:{id}`	30 sec	String (SET NX)
BullMQ queues	`bull:{queueName}:*`	Managed by BullMQ	Stream/List

Key Naming Convention:

```
{domain}:{entity}:{identifier}:{optional_sub_key}

Examples:
tenant:acme                          → tenant slug lookup
tenant:uuid:f1a2b3-...               → tenant UUID lookup
user:uuid:f1a2b3-...                 → user metadata
session:sess_abc123                  → active session
perm:user_uuid:tenant_uuid           → user permissions
jwt:blacklist:jti_abc123             → revoked JWT JTI
refresh:user_uuid:token_id           → refresh token
ratelimit:ip:192.168.1.1:POST:/login → rate limit counter
flags:tenant_uuid                    → feature flags
cache:tenant_uuid:/api/employees:md5hash → cached response
lock:invoice:invoice_uuid            → distributed lock
```

Redis Architecture:



2.6 Authentication & Authorization

JWT Token Design

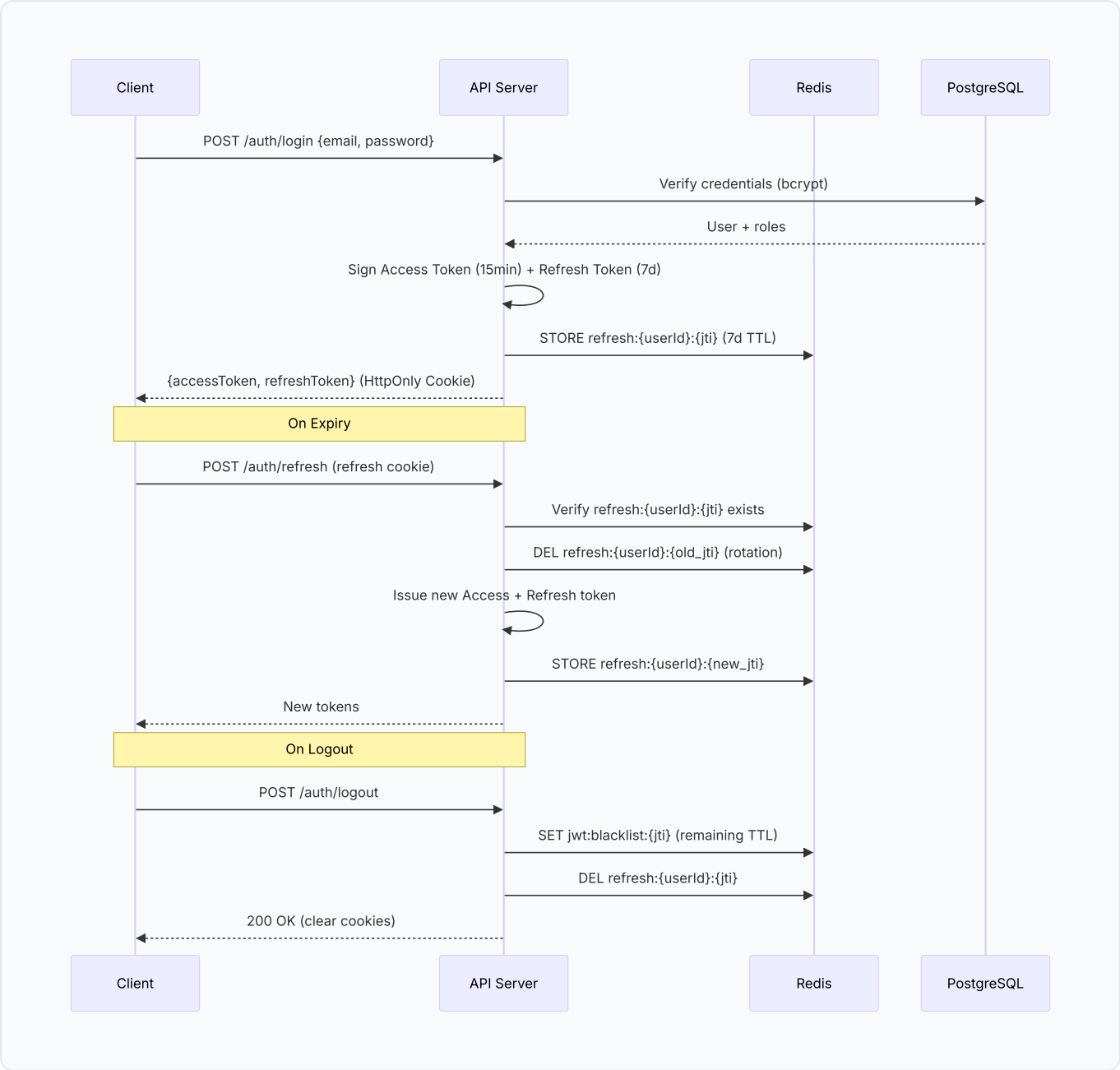
Access Token Payload:

```
{
  "sub": "user-uuid",
  "tid": "tenant-uuid",
  "tsl": "acme",
  "email": "user@acme.com",
  "roles": ["HRMS_ADMIN"],
  "type": "access",
  "jti": "unique-token-id",
  "iat": 1700000000,
  "exp": 1700003600
}
```

Refresh Token Payload:

```
{
  "sub": "user-uuid",
  "tid": "tenant-uuid",
  "jti": "refresh-unique-id",
  "type": "refresh",
  "iat": 1700000000,
  "exp": 1700604800
}
```

Token Flow:



RBAC Model

```
User → User_Roles → Roles → Role_Permissions → Permissions
                                                    ↓
                                          module:action:resource
                                          e.g., hr:employees:read
                                               finance:reports:export
                                               pos:sales:void
```

Permission Check Flow (Middleware)

```
// Middleware example
async function authorize(module, action) {
  return async (req, res, next) ⇒ {
    const cacheKey = `perm:${req.user.id}:${req.tenant.id}`;
    let permissions = await redis.get(cacheKey);

    if (!permissions) {
      permissions = await db.getUserPermissions(req.user.id, req.tenant.schemaName);
      await redis.setex(cacheKey, 1800, JSON.stringify(permissions));
    } else {
      permissions = JSON.parse(permissions);
    }

    const hasPermission = permissions.some(
      p ⇒ p.module ≡≡ module && p.action ≡≡ action
    );

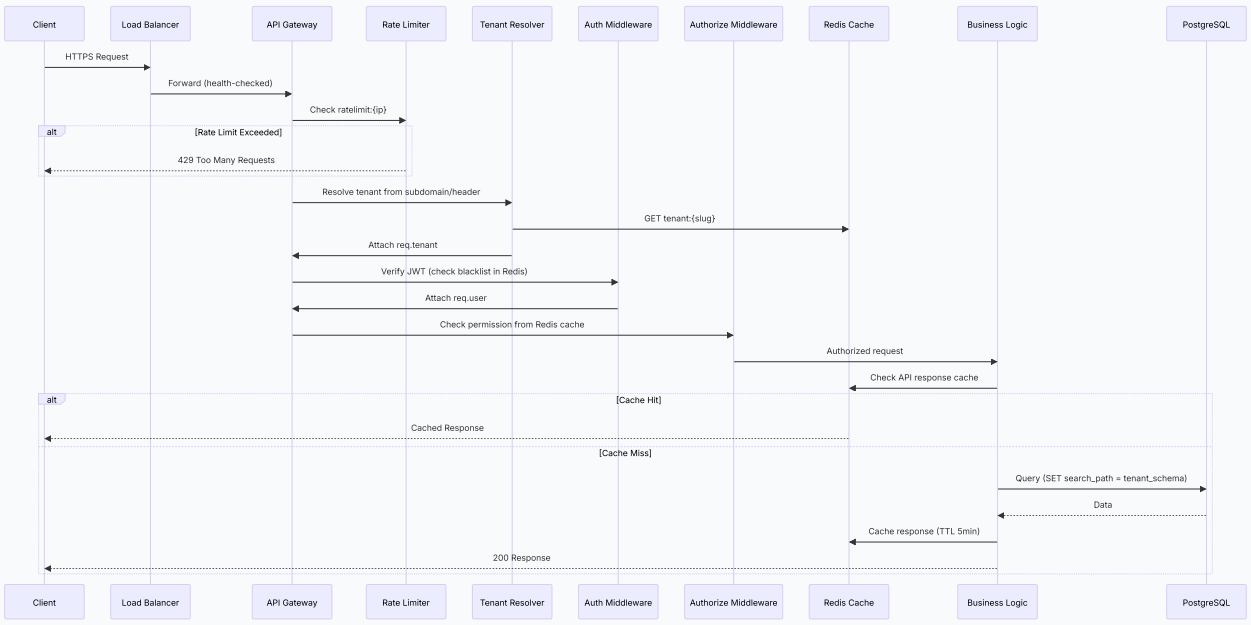
    if (!hasPermission) return res.status(403).json({ error: 'Forbidden' });
    next();
  };
}
```

2.7 Backend Folder Structure

```
src/
├── api/
│   ├── controllers/         # Route handlers (thin layer)
│   │   ├── auth.controller.js
│   │   ├── employee.controller.js
│   │   └── ...
│   ├── routes/              # Express router definitions
│   │   ├── auth.routes.js
│   │   ├── hr.routes.js
│   │   └── index.js
│   └── validators/          # Joi/Zod request validation schemas
│
├── services/                # Business logic layer
│   ├── auth.service.js
│   ├── employee.service.js
│   ├── tenant.service.js
│   └── ...
│
├── repositories/            # Database access layer (no business logic)
│   ├── user.repository.js
│   ├── tenant.repository.js
│   └── base.repository.js
│
├── middlewares/
│   ├── auth.middleware.js     # JWT verification
│   ├── tenant.middleware.js   # Tenant resolution
│   ├── authorize.middleware.js # RBAC enforcement
│   ├── rateLimit.middleware.js # Redis rate limiting
│   ├── errorHandler.middleware.js
│   └── requestLogger.middleware.js
│
├── tenants/
│   ├── tenant.resolver.js    # Subdomain / header / domain resolution
│   ├── tenant.provisioner.js # Create schema, seed roles on new tenant
│   └── tenant.cache.js       # Redis helpers for tenant caching
│
├── auth/
│   ├── jwt.js                # Sign / verify tokens
│   ├── tokenStore.js         # Refresh token Redis storage
│   ├── blacklist.js          # JWT blacklist helpers
│   └── password.js           # bcrypt helpers
│
├── cache/
│   ├── redis.client.js       # Redis connection
│   ├── cache.helper.js       # GET / SET / DEL wrappers
│   ├── permission.cache.js
│   └── tenant.cache.js
│
├── database/
│   ├── pg.client.js          # pg Pool setup
│   ├── pgbouncer.config.js
│   ├── migrations/           # db-migrate or Flyway scripts
│   │   ├── public/           # Master schema migrations
│   │   └── tenant/           # Per-tenant schema template
│   └── seeds/
```

```
│
├── queues/
│   ├── bullmq.client.js      # BullMQ connection
│   ├── queues/
│   │   ├── email.queue.js
│   │   ├── report.queue.js
│   │   ├── notification.queue.js
│   │   └── audit.queue.js
│   └── producers/            # Job producers used by services
│
├── workers/
│   ├── email.worker.js
│   ├── report.worker.js
│   ├── notification.worker.js
│   ├── cleanup.worker.js
│   └── audit.worker.js
│
├── events/
│   ├── eventBus.js           # In-process EventEmitter (or pub/sub)
│   └── handlers/
│
├── utils/
│   ├── logger.js             # Pino structured logger
│   ├── pagination.js
│   ├── queryBuilder.js       # Safe parameterized query builder
│   ├── encryption.js
│   └── response.js           # Standardized API response shapes
│
├── config/
│   ├── env.js                # Validated env vars (Zod schema)
│   ├── redis.config.js
│   ├── pg.config.js
│   ├── jwt.config.js
│   └── bullmq.config.js
│
├── monitoring/
│   ├── prometheus.js         # Metrics exporter
│   ├── healthCheck.js
│   └── tracing.js            # OpenTelemetry
│
└── app.js                    # Express app setup
    server.js                 # HTTP server entry
```

2.8 Request Lifecycle



2.9 Caching Strategy

What to Cache:

Data	Key	TTL	Invalidation Trigger
Tenant metadata	`tenant:{slug}`	1 hour	Tenant settings update
User permissions	`perm:{uid}:{tid}`	30 min	Role assignment change
API responses (read-heavy)	`cache:{tid}:{route}:{hash}`	5 min	Data write events
Feature flags	`flags:{tid}`	5 min	Settings change
Module access	`modules:{tid}`	1 hour	Subscription change

Cache Invalidation Strategy:

```
// Event-driven invalidation
eventBus.on('tenant.settings.updated', async ({ tenantSlug, tenantId }) => {
  await redis.del(`tenant:${tenantSlug}`);
  await redis.del(`flags:${tenantId}`);
  await redis.del(`modules:${tenantId}`);
});

eventBus.on('user.role.changed', async ({ userId, tenantId }) => {
  await redis.del(`perm:${userId}:${tenantId}`);
});
```

Cache Warming on Startup:

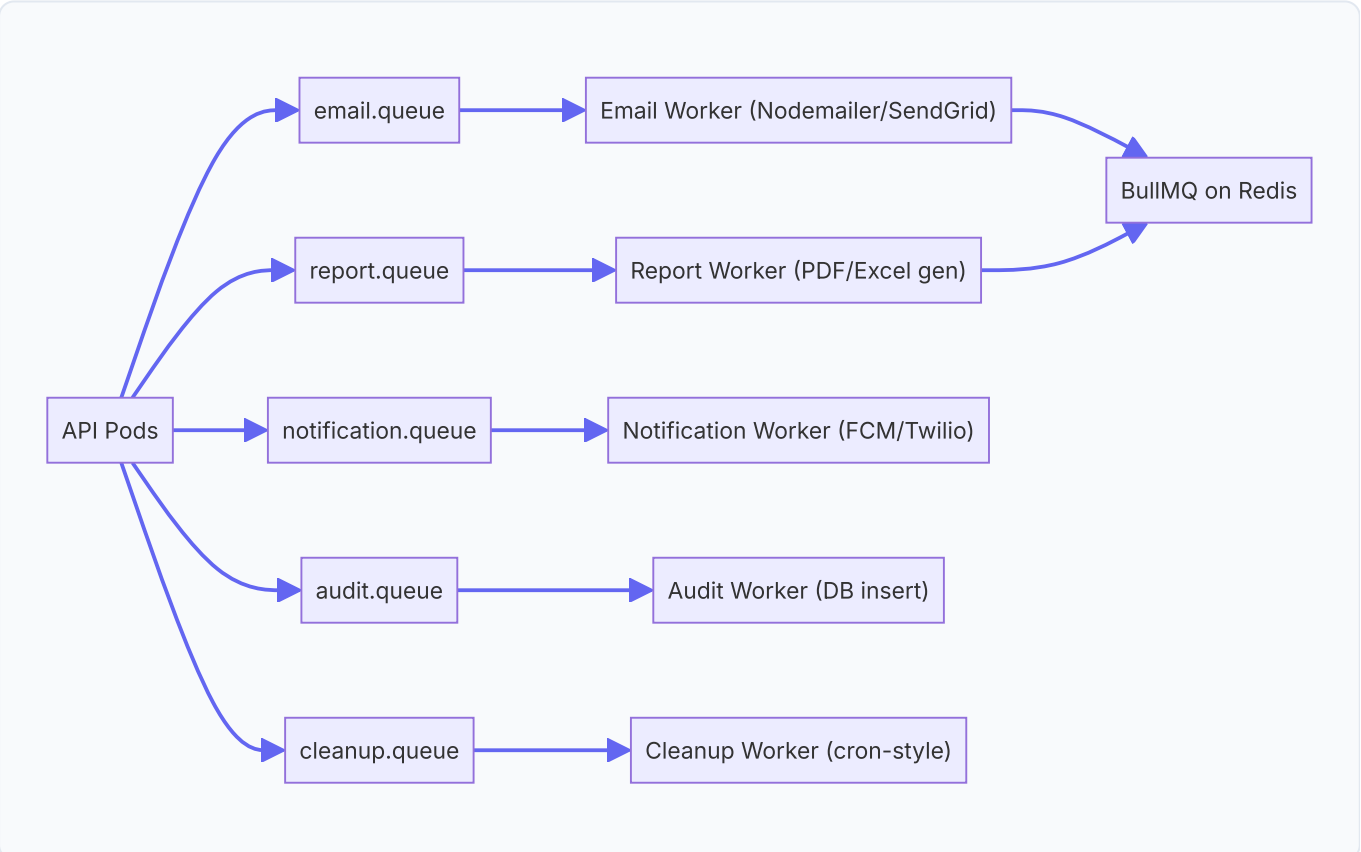
- Load all active tenant slugs into Redis on pod startup
- Pre-warm system roles and permissions

Tenant Isolation in Redis:

- All keys are prefixed with `tenant:{id}` or `perm:{uid}:{tid}`
- Redis ACL rules restrict worker access to specific key namespaces
- Separate Redis logical databases per environment (not per tenant — use key namespacing)

2.10 Background Jobs

Queue Architecture:



BullMQ Queue Definitions:

Queue	Jobs	Priority	Retry
`email`	Welcome, password reset, alerts	Medium	3x exp backoff
`report`	Monthly payroll, VAT, inventory reports	Low	2x
`notification`	In-app, push, SMS	High	5x
`audit`	All write operations	Low	5x
`cleanup`	Expired sessions, temp files	Low	1x
`import`	Bulk data imports (CSV/Excel)	Low	2x
`scheduled`	Daily/weekly automated reports	Cron	3x

```
// Producer example
import { emailQueue } from '../queues/queues/email.queue.js';

await emailQueue.add('welcome-email', {
  to: user.email,
  tenantId: tenant.id,
  template: 'welcome',
  data: { name: user.name, loginUrl: `https://${tenant.slug}.emirateserp.com` }
}, {
  attempts: 3,
  backoff: { type: 'exponential', delay: 2000 },
  removeOnComplete: 100,
  removeOnFail: 500
});
```

2.11 Security

Row-Level Security (RLS)

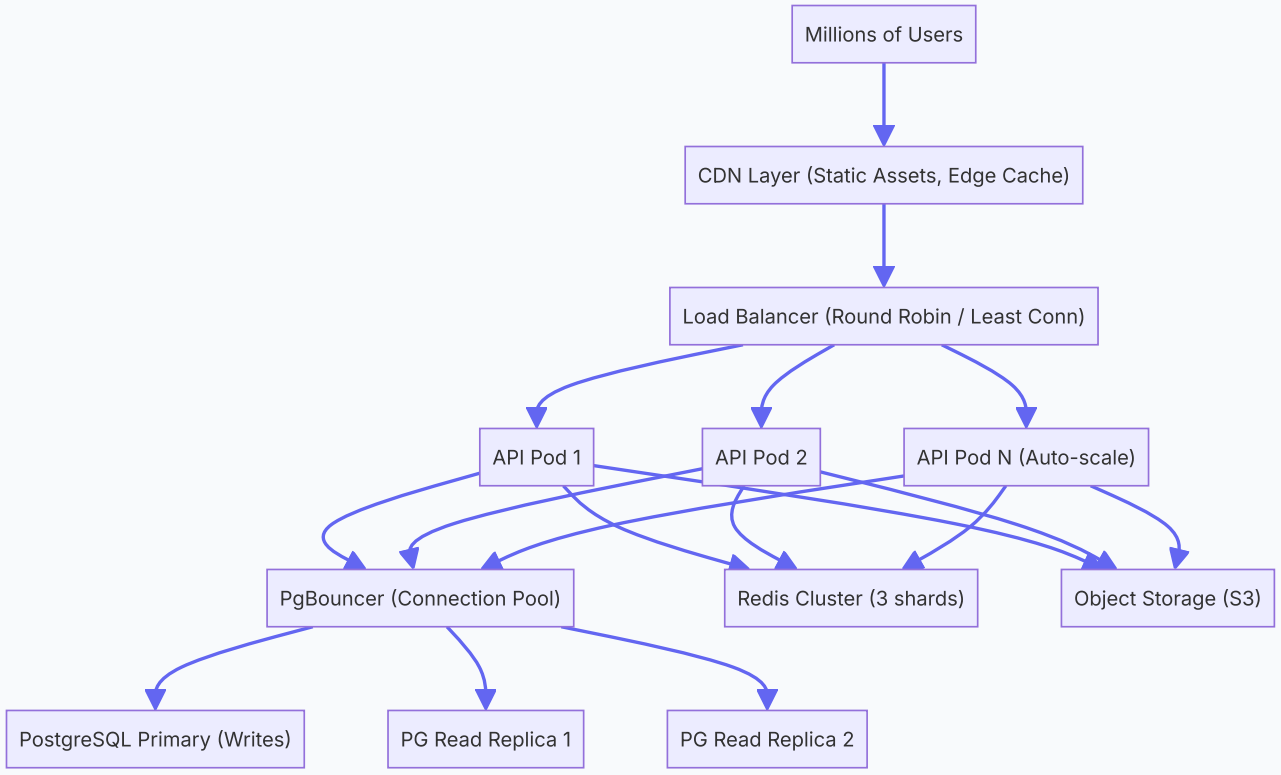
```
-- Set tenant context before each query
SET LOCAL app.current_tenant_id = '{tenantId}';

-- Policy ensures no cross-tenant leakage
CREATE POLICY tenant_rls ON employees
  USING (tenant_id = current_setting('app.current_tenant_id')::UUID);
```

Security Checklist

Layer	Control	Implementation
Transport	TLS 1.3	Nginx / Load Balancer
Auth	JWT HttpOnly Cookies	`sameSite: Strict, secure: true`
Password	bcrypt rounds=12	`bcrypt.hash(password, 12)`
SQL Injection	Parameterized queries only	`pg` library `$1` params
XSS	CSP headers + output encoding	Helmet.js
CSRF	SameSite cookies + CSRF token	csurf or double-submit
Rate Limiting	Redis sliding window	`rate-limiter-flexible`
Secrets	Env vars from vault	HashiCorp Vault / AWS Secrets Manager
Tenant Isolation	Schema + RLS	PostgreSQL
CORS	Whitelist tenant origins	Per-tenant allowed origins list
API Keys	SHA-256 hashed in DB	Prefix-based lookup
Audit Logging	All mutations logged	BullMQ audit queue
JWT Security	Short-lived (15min) + Refresh rotation	JTI tracking in Redis
Encryption	AES-256 for PII at rest	Node.js crypto

2.12 Scaling Strategy

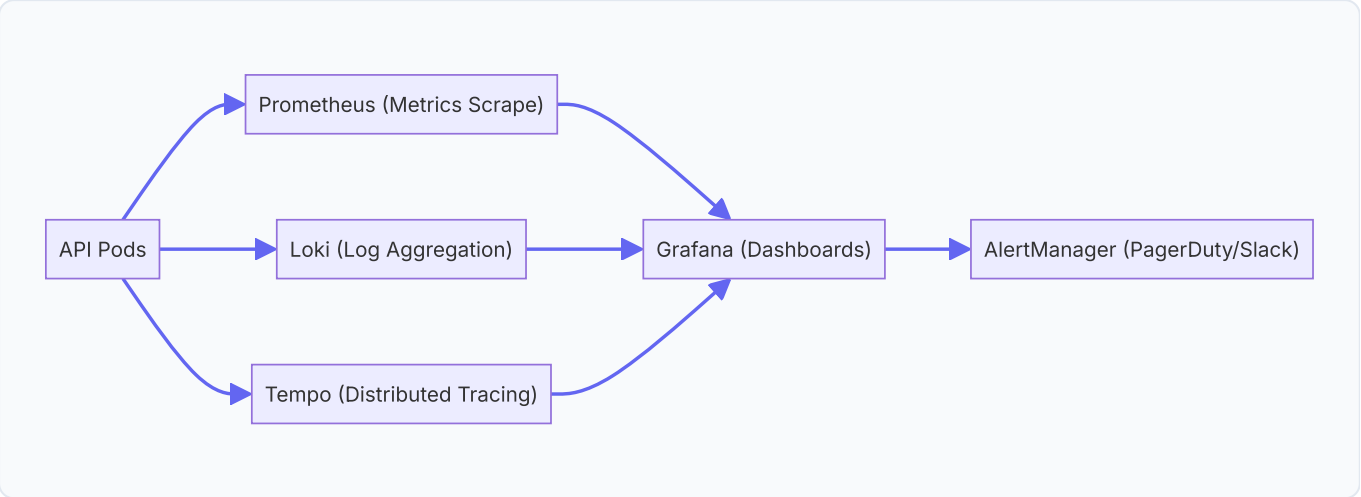


Scaling Tactics:

Tactic	Detail
Stateless API Pods	No session state in process — all in Redis
Horizontal Pod Autoscaling	K8s HPA based on CPU/memory
Read Replicas	Route SELECTs to replicas, writes to primary
PgBouncer	Connection pooling (transaction mode) to PostgreSQL
Redis Cluster	3 shards with replicas for horizontal Redis scaling
CDN	Static files, public assets, country-level caching
DB Partitioning	Audit logs partitioned by month
Schema isolation	Enables future shard-per-tenant migration

2.13 Monitoring

Monitoring Stack:



Key Metrics to Instrument:

```
// Custom metrics
const httpRequestDuration = new promClient.Histogram({
  name: 'http_request_duration_ms',
  help: 'Duration of HTTP requests in ms',
  labelNames: ['method', 'route', 'status_code', 'tenant_id'],
  buckets: [5, 10, 25, 50, 100, 250, 500, 1000, 2500]
});

const activeTenantsGauge = new promClient.Gauge({
  name: 'active_tenants_total',
  help: 'Number of active tenants'
});

const queueJobsGauge = new promClient.Gauge({
  name: 'bullmq_queue_jobs_waiting',
  help: 'Jobs waiting in BullMQ queues',
  labelNames: ['queue']
});
```

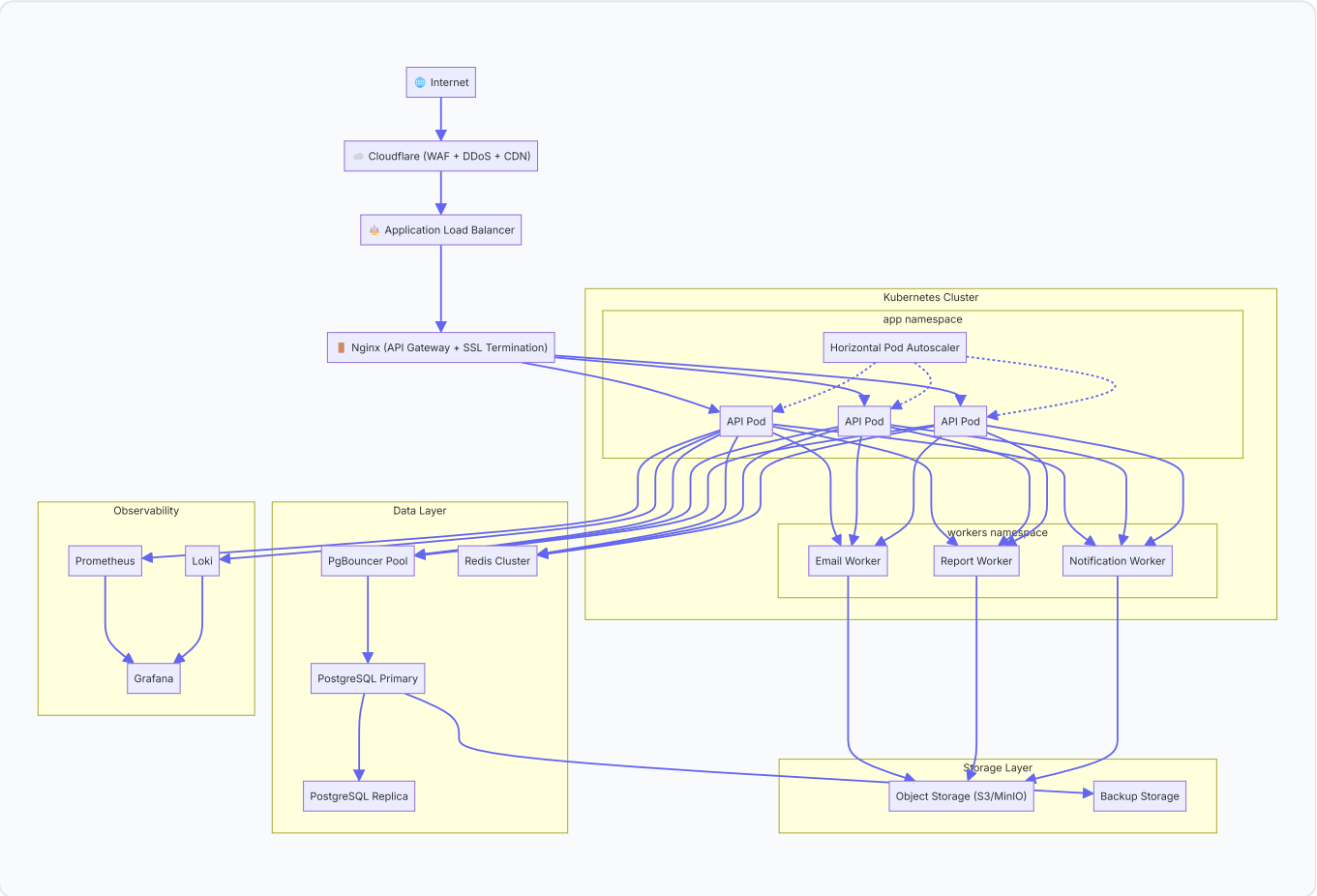
Health Check Endpoint:

```
app.get('/health', async (req, res) => {
  const checks = {
    postgres: await checkPostgres(),
    redis: await checkRedis(),
    workers: await checkWorkers(),
  };
  const healthy = Object.values(checks).every(c => c.status === 'ok');
  res.status(healthy ? 200 : 503).json({ status: healthy ? 'healthy' : 'degraded', checks });
});
```

Structured Logging (Pino):

```
logger.info({
  event: 'user.login',
  userId: user.id,
  tenantId: tenant.id,
  ip: req.ip,
  duration: Date.now() - start,
}, 'User logged in successfully');
```

2.14 Deployment Architecture



2.15 CI/CD Recommendations

Git Branching Strategy (GitFlow):

```
main          → production (protected, requires PR + 2 reviews)
staging       → staging environment (auto-deploy on merge)
develop       → integration branch
feature/*     → individual features
hotfix/*      → production patches
```

GitHub Actions Pipeline:

```
name: CI/CD Pipeline

on:
  push:
    branches: [main, staging, develop]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run lint
      - run: npm run test:unit
      - run: npm run test:integration  # Uses test DB

  build:
    needs: test
    steps:
      - run: docker build -t emirateserp/api:${{ github.sha }} .
      - run: docker push emirateserp/api:${{ github.sha }}

  migrate:
    needs: build
    if: github.ref == 'refs/heads/main'
    steps:
      - run: npm run db:migrate:prod  # Flyway / db-migrate

  deploy:
    needs: migrate
    steps:
      - run: kubectl set image deployment/api api=emirateserp/api:${{ github.sha }}
      - run: kubectl rollout status deployment/api
```

Zero-Downtime Deployment:

```
# Kubernetes rolling update strategy
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0  # Never take a pod down before new one is ready
```

Database Migration Strategy:

- Use backward-compatible migrations (additive changes first)
- Never drop columns in migration — add nullable, backfill, then drop in next release
- Run migrations before pod rollout (init container or pre-deploy job)

Rollback Strategy:

```
kubectl rollout undo deployment/api  # Instant rollback to previous image
```

2.16 Production Best Practices (50+)

Node.js

1. Use `cluster` module or PM2 to utilize all CPU cores
2. Always use `async/await` with proper `try/catch`; never leave unhandled promise rejections
3. Validate all inputs with Zod or Joi before processing
4. Use Pino for structured JSON logging — never `console.log` in production
5. Set `NODE_ENV=production` for Express optimizations
6. Implement graceful shutdown: drain connections, close DB pools on SIGTERM
7. Use Helmet.js for security headers (CSP, HSTS, X-Frame-Options)
8. Never commit secrets — use environment variables validated at startup
9. Implement `requestId` (UUID) for every request for distributed tracing
10. Use `compression` middleware for gzip on responses
11. Set meaningful `timeout` on all outbound HTTP calls
12. Use `express-validator` or Zod for route-level validation
13. Return standardized error shapes `{ error, code, requestId }`
14. Use `http-terminator` for graceful HTTP server shutdown
15. Configure `trust proxy` when behind Nginx/load balancer

PostgreSQL

16. Always use parameterized queries — never string interpolation
17. Use UUIDs (`gen_random_uuid()`) as primary keys for horizontal scalability
18. Index foreign keys, frequently-filtered columns, and JSONB paths
19. Use `EXPLAIN ANALYZE` regularly to detect slow queries
20. Partition large tables (audit_logs, events) by time range
21. Set `statement_timeout` per connection to prevent runaway queries
22. Use `pg_stat_statements` to identify top slow queries in production
23. Enable `idle_in_transaction_session_timeout` to prevent connection leaks
24. Use `JSONB` for flexible schemas but index queried keys with `GIN`
25. Use connection pooling (PgBouncer) — never direct pool from each pod
26. Run `VACUUM ANALYZE` regularly; enable `autovacuum` properly
27. Use `RETURNING *` to avoid extra SELECT after INSERT/UPDATE
28. Set `max_connections` in PostgreSQL conservatively; rely on PgBouncer
29. Enable SSL/TLS for PostgreSQL connections

30. Use `COPY` command for bulk inserts instead of row-by-row INSERT

Redis

31. Always set TTL on every key — never create keys without expiry
32. Use Redis ACL to restrict key access per service
33. Implement circuit breaker: if Redis is down, fall through to DB
34. Use `SCAN` instead of `KEYS *` in production
35. Use Lua scripts for atomic multi-step operations
36. Monitor `used_memory` and `eviction_policy` (`allkeys-lru` recommended)
37. Use `SETNX` (SET NX PX) for distributed locks
38. Separate Redis instances for cache vs. BullMQ queues vs. sessions
39. Never store sensitive PII unencrypted in Redis
40. Enable Redis persistence (`AOF + RDB`) for queue data

Multi-Tenant SaaS

41. Never trust tenant identity from client alone — resolve server-side
42. Enforce schema search path (`SET search_path = tenant_schema`) in every query
43. Implement per-tenant rate limits, not just per-IP
44. Validate tenant subscription/plan before serving premium features
45. Log all cross-tenant access attempts as security events
46. Implement tenant onboarding as a transactional provisioning job
47. Use schema migrations templates for consistent tenant schema versioning
48. Audit every data export request per tenant for GDPR compliance
49. Implement tenant data deletion (right to erasure) as a background job
50. Test tenant isolation in integration tests: assert tenant A cannot see tenant B data

DevOps / Security

51. Rotate JWT signing keys periodically without downtime (key versioning in token header)
52. Store secrets in HashiCorp Vault or cloud secret manager — not `.env` files
53. Implement IP allowlisting for admin endpoints
54. Run Dependabot / Snyk for dependency vulnerability scanning
55. Use `HTTPS only` with HSTS max-age of 1 year
56. Set `Content-Security-Policy` headers to prevent XSS
57. Implement anomaly detection: flag accounts with >10 failed logins/minute
58. Run penetration tests before each major release
59. Use feature flags (Redis-backed) for gradual feature rollouts
60. Implement dead-letter queues in BullMQ for failed job inspection

---

Design Decisions

Decision	Chosen Approach	Rationale
Multi-tenancy model	Schema per tenant	Balance of isolation, cost, and scalability
Auth storage	HttpOnly Cookies	XSS immunity vs. localStorage
Token strategy	Short-lived JWT + Redis-backed refresh rotation	Security + scalability
Caching	Redis with per-tenant key namespacing	Speed + isolation
Job queue	BullMQ on Redis	Simple, reliable, retries, dashboards
Logging	Pino (structured JSON)	Performance, parseability
Connection pooling	PgBouncer (transaction mode)	Handle thousands of connections
RLS	Enabled on all tenant tables	Defense-in-depth against cross-tenant leakage

Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Schema migration failure across N tenants	Medium	High	Transactional migrations + dry-run checks
Redis failure	Low	High	Circuit breaker + fallback to DB
PostgreSQL primary failure	Low	Critical	Failover to replica with pgPool-II or Patroni
JWT secret compromise	Low	Critical	Secret rotation + short TTL
Tenant data leakage via bug	Low	Critical	RLS + integration tests
BullMQ queue saturation	Medium	Medium	Queue monitoring + autoscale workers

Future Improvements

1. **Microservices Migration** — Extract HR, Finance, POS, CRM into separate services behind API Gateway
2. **GraphQL Federation** — Unified GraphQL schema across microservices
3. **Event Sourcing** — Use Apache Kafka for audit, analytics, and event-driven integrations
4. **Database per Large Tenant** — Shard top 1% tenants to dedicated PostgreSQL clusters
5. **SSO Integration** — SAML 2.0 / OIDC for enterprise tenants
6. **AI/ML Pipeline** — Analytics service using tenant-isolated data lake
7. **Edge Caching** — Cloudflare Workers for tenant resolution at edge
8. **WASM-based Reporting** — Run report generation in browser to reduce server load

---

Document generated by Senior Architecture Review — Emirates ERP Platform
Revision history should be tracked in source control.